

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Mock Interview (Low)
Assessment Tool Weightage: 30%

Dr. R. Kesavan

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'	BL3 – Apply	5
2	Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'	BL4 – Analyze	5
3	Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.'	BL3 – Apply	5
4	Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'	BL3 – Apply	5
5	Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?'	BL4 – Analyze	5
6	Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'	BL4 – Analyze	5
7	Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'	BL4 – Analyze	5

8	Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'	BL3 – Apply	5
9	Interviewer: 'What is a serializability schedule? How do you test for conflict serializability using a precedence graph?'	BL4 – Analyze	5
10	Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?'	BL4 – Analyze	5

Code of Conduct:

I R. Indhu (192524332) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

R. Indhu

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated

Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

Q1. Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?

Answer:

Query processing converts an SQL query into a result. The main steps are:

1. **Parsing** – Checks the syntax of the SQL query.
2. **Translation** – Converts SQL into a query tree or relational algebra.
3. **Optimization** – Generates different execution plans and estimates their costs.
4. **Execution** – Executes the selected plan.
5. **Result** – Returns the output to the user.

The query optimizer selects the plan with the **lowest estimated cost**, considering factors such as disk I/O, CPU time, memory, indexes, and join methods.

Example:

SQL Query



Parsing



Query Tree



Optimization



Best Execution Plan



Execution



Result

Short answer:

The optimizer compares possible execution plans and chooses the one with the lowest estimated cost.

Q2. Difference between heuristic optimization and cost-based optimization

Answer:

Heuristic optimization uses predefined rules to improve a query, while **cost-based**

optimization calculates the estimated cost of different execution plans and selects the cheapest one.

Heuristic	Cost-Based
Uses rules	Uses cost estimates
Faster optimization	More detailed optimization
Does not compare all plans	Compares multiple plans
Simple queries	Complex queries

Example:

A heuristic rule can be:

Perform selection before joining tables to reduce the number of records.

Cost-based optimization may compare:

Plan 1 → Cost = 500

Plan 2 → Cost = 300

Plan 3 → Cost = 700

It selects **Plan 2**.

Conclusion:

I would use heuristic optimization for simple and quick decisions, and cost-based optimization for complex queries and large databases.

Q3. How would you implement database connectivity? Difference between JDBC and ODBC.

Answer:

In a real project, I would use the following steps:

1. Load the database driver.
2. Create a database connection.
3. Create a statement or prepared statement.
4. Execute the SQL query.
5. Process the result.
6. Close the resources.

JDBC

JDBC stands for **Java Database Connectivity**. It is used by Java applications to connect to databases.

ODBC

ODBC stands for **Open Database Connectivity**. It is a general database connectivity standard that can be used by different programming languages.

JDBC	ODBC
Java-specific	Language-independent standard
Used mainly with Java	Used by many languages
Java API	General database API

Example:

Java Application



JDBC Driver



MySQL Database

Conclusion:

For a Java project, I would normally use JDBC.

Q4. What are ACID properties? Give a real-world example.

Answer:

ACID stands for:

- **A – Atomicity**
- **C – Consistency**
- **I – Isolation**
- **D – Durability**

1. Atomicity

A transaction is completed **fully or not at all**.

Example: In a bank transfer, money must be deducted from one account and credited to another. If one operation fails, the whole transaction is rolled back.

2. Consistency

The database must remain in a **valid state** before and after a transaction.

Example: A bank account should not have an invalid balance.

3. Isolation

Multiple transactions should not interfere with each other.

Example: Two people withdrawing money at the same time should not cause incorrect balance calculations.

4. Durability

Once a transaction is committed, the data remains saved even after a failure.

Example: A completed bank transaction should remain recorded after a power failure.

Easy memory:

A → All or Nothing

C → Correct Data

I → Independent Transactions

D → Data Stays

Q5. Explain dirty read, lost update, and unrepeatable read. How does 2PL solve them?

Answer:

These are common **concurrency control problems**.

1. Dirty Read

A transaction reads data that another transaction has changed but not committed.

T1 → Updates data

T2 → Reads uncommitted data

T1 → Rollback

T2 has read incorrect temporary data.

2. Lost Update

Two transactions update the same data, and one update overwrites the other.

T1 → Reads 100

T2 → Reads 100

T1 → Updates 120

T2 → Updates 150

T1's update is lost.

3. Unrepeatable Read

A transaction reads the same record twice and gets different values because another transaction changed it between the reads.

How 2PL helps

Two-Phase Locking (2PL) uses:

Growing Phase → Acquire locks

Shrinking Phase → Release locks

Locks prevent unsafe concurrent operations and ensure **conflict serializability**.

Q6. What is deadlock? Explain detection and prevention.

Answer:

A **deadlock** occurs when two or more transactions wait for each other indefinitely.

Example:

T1 holds Resource A

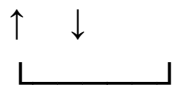
T2 holds Resource B

T1 waits for B

T2 waits for A

So:

T1 → T2



This is a cycle.

Deadlock Detection

We use a **Wait-For Graph (WFG)**.

- Transactions are represented as nodes.
- An edge $T1 \rightarrow T2$ means T1 is waiting for T2.
- If the graph contains a **cycle**, there is a deadlock.

Prevention strategies

1. Resource Ordering

Always request resources in a fixed order.

2. Wait-Die

An older transaction may wait, while a younger transaction may be rolled back depending on the timestamp rules.

3. Wound-Wait

An older transaction can force a younger transaction to roll back.

Conclusion

Deadlocks can be detected using cycles in a Wait-For Graph and prevented using techniques such as resource ordering, wait-die, and wound-wait.

Q7. Compare Immediate Update and Deferred Update recovery.

Answer:

Immediate Update

In immediate update, database changes can be written **before the transaction commits**.

Therefore, after a crash, the system may need:

UNDO + REDO

Deferred Update

In deferred update, database changes are kept until the transaction commits.

After a crash, recovery mainly requires:

REDO of committed transactions

Comparison

Feature	Immediate Update	Deferred Update
Database update before commit	Yes	No
UNDO	Required when needed	Generally not required
REDO	May be required	Required
Recovery	More complex	Simpler

When to use?

Immediate update is useful when changes need to reach the database during transaction execution. Deferred update is preferred when simpler recovery and delayed database writes are acceptable.

Easy memory:

Immediate → UNDO + REDO

Deferred → REDO

Q8. Explain Shadow Paging. Advantages and limitations.

Answer:

Shadow Paging is a database recovery technique that maintains two page tables:

1. Shadow Page Table

2. Current Page Table

The shadow page table stores the stable version of the database. Changes are made using the current page table.

Working:

Shadow Page Table



Stable Data

Current Page Table



Updated Data

If the transaction commits:

Current → New stable version

If failure occurs:

Failure



Discard Current Table



Use Shadow Table

Advantages

- Simple recovery.
- No complex log processing.
- Easy rollback.

Limitations

- Extra page management is required.
- Can cause fragmentation.
- Not ideal for very large and frequently updated databases.

Conclusion

Shadow paging is simple and useful for recovery, but log-based recovery is generally more flexible for large and heavily updated systems.

Q9. What is a serializability schedule? How do you test conflict serializability?

Answer:

A **serializable schedule** is a concurrent transaction schedule whose result is equivalent to executing the transactions one after another in some serial order.

To test **conflict serializability**, we create a **precedence graph**.

Steps:

1. Create one node for each transaction.
2. Find conflicting operations.
3. If transaction T1 executes a conflicting operation before T2, draw:

$T1 \rightarrow T2$

4. Check the graph for a cycle.

Example:

$T1 \rightarrow T2$

$T2 \rightarrow T3$

There is **no cycle**, so it is conflict serializable.

But:

$T1 \rightarrow T2$

$T2 \rightarrow T1$

contains a cycle.

Therefore:

No cycle $\rightarrow$ Conflict Serializable

Cycle $\rightarrow$ Not Conflict Serializable

Q10. Basic 2PL, Strict 2PL and Rigorous 2PL

Answer:

All three are variations of the **Two-Phase Locking protocol**.

1. Basic 2PL

It has two phases:

Growing $\rightarrow$ Acquire locks

Shrinking $\rightarrow$ Release locks

After releasing a lock, the transaction cannot acquire a new lock.

2. Strict 2PL

All **exclusive (X) locks** are held until the transaction commits or aborts.

This prevents many cascading rollback problems.

3. Rigorous 2PL

All **locks**, both shared and exclusive, are held until the transaction commits or aborts.

Comparison

Feature	Basic 2PL	Strict 2PL	Rigorous 2PL
Growing phase	Yes	Yes	Yes
Shrinking phase	Yes	At commit/abort for X locks	At commit/abort for all locks
X-lock until commit	No	Yes	Yes
S-lock until commit	No	Not necessarily	Yes
Recovery	More complex	Easier	Very simple

Which is commonly used?

Strict 2PL is commonly used because it provides conflict serializability, prevents dirty updates and cascading rollback problems, and makes recovery easier.